

세미나 2회차: 경로 계획과 제어

육시우

2026-06-22



ACE Lab.



ACE Lab.

KNU

목차

- 1. 그래프 탐색과 최단 경로 알고리즘
- 2. ROS2 Navigation Stack 구조 이해
- 3. 로컬 경로 계획
- 4. PID 제어

1-1. 경로 계획이란 무엇인가

로봇이 시작점(Start)에서 목표점(Goal)까지 이동할 때, **충돌 없이 가장 효율적인 경로를 찾는 과정**

구분	설명
전역(Global) 경로 계획	전체 지도 기반의 최적 경로 탐색
지역(Local) 경로 계획	주변 장애물 회피 중심의 실시간 경로 보정

1-2. 그래프 탐색

경로 계획 문제는 보통 **그래프(graph)** 로 모델링한다.

용어	의미
노드(Node)	한 위치 또는 지점
엣지(Edge)	두 노드 간의 연결
비용(Cost)	이동 거리 또는 시간

1-3. 다익스트라 알고리즘

- 모든 노드까지의 최소 비용 경로를 찾는 알고리즘. 네비게이션에서 목적지까지 가는 빠른 길을 찾거나, 네트워크에서 패킷을 전송할 때 최적의 경로를 설정할 때 사용

특징	설명
탐색 방식	BFS 기반, 누적 비용 최소화
장점	음이 아닌 비용에서 항상 최적 경로 보장
단점	목표가 멀거나 불필요한 탐색이 많음

1-3. 다익스트라 알고리즘

1. **초기화:** 시작 노드의 거리는 0으로, 나머지 모든 노드의 거리는 무한대(∞)로 설정.
2. **방문하지 않은 노드 탐색:** 아직 방문하지 않은 노드 중 시작점으로부터의 거리가 가장 짧은 노드를 선택.
3. **거리 갱신:** 선택한 노드를 거쳐서 이웃 노드로 가는 비용을 계산. 이 비용이 기존에 기록된 이웃 노드의 최단 거리보다 작다면, 값을 더 작은 값으로 갱신.
4. **반복:** 모든 노드를 방문할 때까지 2번과 3번 과정을 반복.

1-4. A* 알고리즘

- 다익스트라 알고리즘에 휴리스틱(Heuristic, 추정치)이라는 개념을 더한 알고리즘. 즉, 최종 목적지가 어느 방향에 있는지 알고 그쪽을 우선적으로 탐색.

요소	설명
$g(n)$	시작점부터 현재 노드까지의 실제 비용
$h(n)$	휴리스틱 (목표까지의 예상 거리)
$f(n)$	총 비용 = $g(n) + h(n)$

$g(n)$: 출발지에서 현재 노드 n 까지 오는데 걸린 **실제 비용** (다익스트라의 개념)

$h(n)$: 현재 노드 n 에서 목적지까지 갈 것으로 예상되는 **추정 비용** (휴리스틱)

1-5. 알고리즘 비교

항목	Dijkstra	A*
탐색 기준	누적 거리	거리 + 휴리스틱
속도	느림	빠름
정확도	항상 최적	휴리스틱에 따라 달라짐
응용 예시	지도 전체 탐색	자율주행, SLAM, 게임 AI

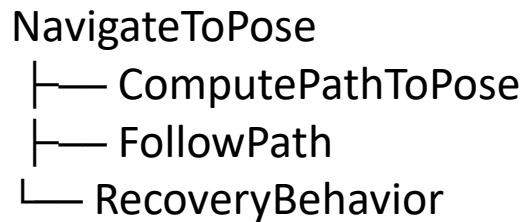
2-1. Navigation Stack 이란?

ROS2 Navigation Stack(줄여서 **Nav2**)은 로봇이 주어진 지도 상에서 스스로 위치를 추정하고, 경로를 계획하고, 목표까지 이동하도록 하는 ROS2의 패키지

구성 요소	노드 이름	주요 역할
Localization	<code>amcl</code>	Particle Filter 기반 위치 추정
Global Planner	<code>nav2_planner</code>	전역 경로 계산 (A*, Dijkstra)
Local Planner	<code>nav2_controller</code>	장애물 회피 중심의 실시간 경로 수정
Costmap	<code>costmap_2d</code>	이동 가능한 영역과 장애물 정보를 지도화
BT Navigator	<code>bt_navigator</code>	Behavior Tree 기반 자율주행 흐름 제어
Lifecycle Manager	<code>lifecycle_manager</code>	Nav2 노드들의 상태 관리

2-2. Behavior Tree (BT) 기반 구조

ROS2 Navigation은 단순한 순차 실행이 아니라, **Behavior Tree**라는 논리 구조로 동작. 예를 들어 다음과 같은 트리 형태로 동작 순서를 정의.



- **ComputePathToPose (경로 생성)**: 지도 상에서 목적지까지 가는 길을 찾는다.
- **FollowPath (이동/경로 추종)**: 생성된 길을 따라 바퀴를 굴려 실제로 이동.
- **RecoveryBehavior (복구 행동/실패 시 재시도)**: 만약 이동 중에 갑자기 사람이 앞을 가로막거나 바퀴가 끼이는 등 '실패' 상황이 오면, 제자리에 서서 한 바퀴 돌며 주변을 다시 살피거나(Recovery) 후진을 하는 등 문제를 해결하려고 시도. 그 후 다시 경로를 만들어 이동.

2-3. 주요 토픽 및 서비스

항목	이름	설명
전역 경로	<code>/plan</code>	Global Planner가 생성한 경로
속도 명령	<code>/cmd_vel</code>	Local Controller가 발행하는 로봇 속도
로봇 위치	<code>/amcl_pose</code>	AMCL에서 추정한 위치
지도 데이터	<code>/map</code>	SLAM 또는 로드된 지도
장애물 정보	<code>/costmap</code>	Costmap이 계산한 이동 비용 지도

3-1. 로컬 경로 계획

전역 경로(Global Path)가 전체 지도 위의 최적 경로라면, 로컬 경로(Local Path)는 실시간 상황 변화에 대응하는 즉각적인 경로.

3-2. ROS2에서 Local Planner의 역할 및 종류

- 로봇의 속도 명령 계산 (cmd_vel)
- 충돌 없는 단기 경로 생성

ROS2의 Nav2에서는 기본적으로 다음 두 가지 로컬 플래너가 사용됩니다.

알고리즘	설명
DWA (Dynamic Window Approach)	속도 공간을 탐색해 충돌 없는 최적 속도 선택
TEB (Timed Elastic Band)	시간 제약 기반의 최적 궤적 계산 (더 부드럽고 정확함)

3-3. DWA

DWA: 로봇의 속도(v, ω) 공간에서 가능한 조합을 탐색하여 다음 순간의 최적 속도를 선택하는 알고리즘

항목	설명
입력	현재 속도, 목표 위치, 장애물 정보
출력	최적의 선속도(v)와 각속도(ω)
핵심 아이디어	"속도 공간에서" 안전하고 효율적인 조합 선택

DWA의 평가 함수:

$$G(v, \omega) = \alpha \cdot heading + \beta \cdot distance + \gamma \cdot velocity$$

- **heading**: 목표 방향과의 일치 정도
- **distance**: 장애물과의 거리
- **velocity**: 속도 크기

3-4. TEB

DWA보다 발전된 형태의 local planner. 경로 전체를 "시간을 포함한 곡선으로 모델링하고 동시에 시간, 속도, 장애물 제약을 최적화한다.

항목	설명
입력	전역 경로, 속도 제한, 장애물 지도
출력	시간 기반의 부드러운 최적 경로
장점	더 안정적이며 실제 로봇의 물리 제약 반영
ROS2 패키지	<code>teb_local_planner</code>

3-5. ROS2 Navigation에서의 로컬 플래너 설정

nav2_params.yaml 파일에서

controller_server:

ros__parameters:

FollowPath:

plugin:

부분을 찾아 코드를 수정하면 로컬 플래너 변경 가능

4. PID 제어

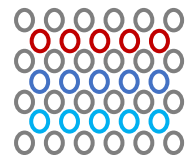
- PID 제어기는 목표값(Setpoint)과 실제값(Feedback)의 차이인 **오차(error)**를 계산하여 이를 **비례(P)**, **적분(I)**, **미분(D)** 항으로 조합하여 제어 입력을 계산
 - P (비례항, Proportional) : "현재"의 오차만 관찰
 - **개념**: 오차가 크면 엑셀을 세게 밟고, 오차가 작으면 살살 밟는 직관적인 방식
 - **역할**: 부족한 속도에 비례해서 **즉각적으로 반응**하여 목표값 근처로 빠르게 끌어올린다.
 - **한계**: 목표값 근처에 도달할수록 오차가 0에 가까워지기 때문에 엑셀을 밟는 힘도 줄어든다. 결국 마찰력이나 무게 때문에 목표값에 딱 도달하지 못하고 살짝 부족한 상태(이를 **정상상태 오차**, **steady-state error**라고 한다)로 계속 달리게 된다.

4. PID 제어

- I (적분항, Integral): "과거"의 오차를 누적
 - **개념:** 아까부터 계속 95km/h에 머물러 있다(목표는 100km/h -> 오차가 계속 쌓이고 있으니 시간 동안 쌓인 오차를 전부 더하는 방식.
 - **역할:** 남아있는 미세한 오차를 전부 더해서 큰 힘을 만들어내므로, 결국 목표값(100km/h)에 딱 맞추도록 장기적 오차를 제거.
- D (미분항, Derivative): "미래"의 변화 추이를 예측
 - **개념:** 목표 속도에 가까워지면서 속도가 너무 빠르게 올라갈 때, 오차가 줄어드는 속도(변화율)를 보고 브레이크를 살짝 밟아주는 방식.
 - **역할:** 목표 속도에 도달할 때 너무 엑셀을 세게 밟아 100km/h를 넘겨버리는 현상(**Overshoot, 오버슈트**)을 막아준다. 급격한 변화를 억제해 부드럽고 안정적으로 정착하게 돕는 댐퍼(완충기) 역할을 한다.

Q & A

Thank you for your attention



Architecture and Compiler
for Embedded Systems Lab.

Graduate School of Electronic and Electrical Engineering, KNU
ACE Lab. (hw051656@gmail.com)